El método de Newton-Raphson en una y varias variables

Aplicaciones en ingeniería mecánica con Python

Whady Felipe Florez

Métodos Numéricos — Ingeniería Mecánica (tercer semestre)

7 de octubre de 2026

Contenido

1. Motivación y planteamiento
2. Newton-Raphson en una variable
3. Newton-Raphson para sistemas
4. Cierre

¿Por qué necesitamos resolver ecuaciones no lineales?

En ingeniería mecánica muchas ecuaciones no se pueden despejar de forma cerrada:

Problema	**Ecuación a resolver**	**Área**
Factor de fricción en tuberías	$\frac{1}{\sqrt{f}} + 2\log_{10}\left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{Re\sqrt{f}}\right) = 0$	Mecánica de fluidos
Frecuencias de una viga en voladizo	$1 + \cos\lambda \cosh\lambda = 0$	Vibraciones
Posición de un mecanismo de cuatro barras	$r_2 e^{i\theta_2} + r_3 e^{i\theta_3} = r_1 + r_4 e^{i\theta_4}$	Cinemática de máquinas
Temperaturas en una aleta con radiación	$kA\,T'' - hP\,(T - T_\infty) - \varepsilon\sigma P\,(T^4 - T_{sur}^4) = 0$	Transferencia de calor

Hoy aprenderemos una sola idea que resuelve las cuatro: el método de Newton-Raphson.

Objetivos de aprendizaje

Al terminar la clase, usted podrá:

- Deducir el método de Newton-Raphson a partir de la serie de Taylor y explicarlo geométricamente.
- Implementarlo en Python para **una** ecuación y para **sistemas** de ecuaciones (matriz Jacobiana).
- Reconocer su convergencia cuadrática y verificarla numéricamente.
- Identificar cuándo falla (semilla mala, $f'(x) = 0$, raíces múltiples, Jacobiana singular).
- Aplicarlo a cuatro problemas de ingeniería mecánica y comparar con `scipy.optimize`.

El problema: encontrar raíces

Planteamiento

Dada $f : \mathbb{R} \to \mathbb{R}$, encontrar x^* tal que $f(x^*) = 0$. (Para sistemas: dada $\mathbf{F} : \mathbb{R}^n \to \mathbb{R}^n$, encontrar $\mathbf{x}^*$ tal que $\mathbf{F}(\mathbf{x}^*) = \mathbf{0}$.)

Métodos cerrados (con intervalo)

- Bisección, falsa posición
- Siempre convergen si hay cambio de signo
- Convergencia lenta (lineal)

Métodos abiertos (con semilla)

- Newton-Raphson, secante
- Convergencia muy rápida
- Pueden fallar: dependen de la semilla x_0

Contenido

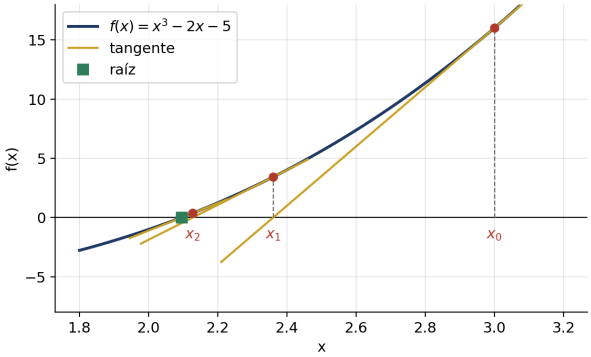
1 Motivación y planteamiento

2 Newton-Raphson en una variable

3 Newton-Raphson para sistemas

4 Cierre

Idea geométrica: seguir la tangente



1. Parta de una semilla x_k.
2. Trace la recta tangente a f en $(x_k, f(x_k))$.
3. Donde la tangente corta el eje x está la nueva aproximación x_{k+1}.

Fórmula de iteración

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Deducción con la serie de Taylor

Aproximamos f cerca de x_k por su polinomio de Taylor de primer orden:

$$f(x) \approx f(x_k) + f'(x_k)\,(x - x_k).$$

Buscamos el punto donde esta aproximación lineal se anula:

$$0 = f(x_k) + f'(x_k)(x_{k+1} - x_k) \quad \Longrightarrow \quad \boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$

Condición necesaria

Se requiere $f'(x_k) \neq 0$. Si $f'(x_k)$ es muy pequeña, el paso $\Delta x = -f/f'$ se vuelve enorme.

Interpretación

Newton reemplaza, en cada paso, un problema *no lineal* difícil por un problema *lineal* fácil.

Algoritmo y criterios de parada

Algoritmo

1. Elegir x_0, tolerancia, $k_{\text{máx}}$.
2. Para $k = 0, 1, 2, \ldots$
   1. $\Delta x_k = -f(x_k)/f'(x_k)$
   2. $x_{k+1} = x_k + \Delta x_k$
   3. Si se cumple el criterio de parada: terminar.
3. Si $k > k_{\text{máx}}$: reportar no convergencia.

Criterios de parada (use más de uno)

- Paso pequeño: $|\Delta x_k| < \text{tol} \cdot \text{máx}(1, |x_{k+1}|)$
- Residuo pequeño: $|f(x_{k+1})| < \text{tol}_f$
- Máximo de iteraciones $k_{\text{máx}}$

Cuidado: un residuo pequeño no garantiza un error pequeño si $|f'|$ es muy pequeña (y viceversa).

Implementación mínima en Python

```python
def newton_1d(f, df, x0, tol=1e-12, maxit=50):
    x = float(x0)
    hist = [x] # guardamos todos los iterados
    for k in range(maxit):
        dx = -f(x) / df(x) # paso de Newton
        x = x + dx
        hist.append(x)
        if abs(dx) < tol * max(1.0, abs(x)):
            break # criterio de parada
    return x, hist
```

- Recibe la función `f` **y su derivada** `df`.
- Devuelve la raíz y el *historial* de iterados (para graficar la convergencia).
- El mismo código servirá en todos los ejemplos de ingeniería de hoy.

Ejemplo de calentamiento: $f(x) = x^2 - 2$, $x_0 = 1$

k	x_k	$\lvert x_k - \sqrt{2}\rvert$
0	1.000000000000000	4.1×10^{-1}
1	1.500000000000000	8.6×10^{-2}
2	1.416666666666667	2.5×10^{-3}
3	1.414215686274510	2.1×10^{-6}
4	1.414213562374690	1.6×10^{-12}

Aquí $x_{k+1} = \frac{1}{2}\left(x_k + \frac{2}{x_k}\right)$ (método de Herón).

Observe

El número de dígitos correctos se duplica aproximadamente en cada iteración:
$1 \to 2 \to 5 \to 11 \to \cdots$

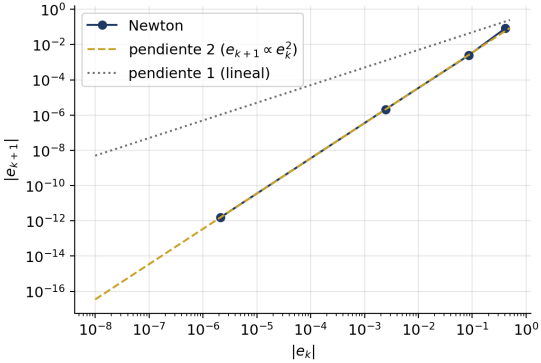
Convergencia cuadrática

Teorema (convergencia local)

Si $f \in C^2$, $f(x^*) = 0$ y $f'(x^*) \neq 0$, existe $\delta > 0$ tal que para $|x_0 - x^*| < \delta$ Newton converge y

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} = \left|\frac{f''(x^*)}{2f'(x^*)}\right|, \qquad e_k = x_k - x^*.$$

Verificación numérica: la pendiente de $\log|e_{k+1}|$ vs. $\log|e_k|$ es ≈ 2.



Ejemplo 1 (Mecánica de fluidos): factor de fricción de Colebrook

Pérdida de carga en una tubería: $\Delta P = f\,\dfrac{L}{D}\,\dfrac{\rho V^2}{2}$. El factor f está implícito:

$$\frac{1}{\sqrt{f}} = -2\log_{10}\left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{Re\sqrt{f}}\right)$$

Cambio de variable $x = 1/\sqrt{f}$ (hace la ecuación casi lineal):

$$F(x) = x + 2\log_{10}\left(\frac{\varepsilon/D}{3.7} + \frac{2.51\,x}{Re}\right) = 0, \qquad F'(x) = 1 + \frac{2}{\ln 10}\,\frac{2.51/Re}{\dfrac{\varepsilon/D}{3.7} + \dfrac{2.51\,x}{Re}}.$$

Datos

Agua en tubería de acero comercial: $D = 50$ mm, $\varepsilon = 0.045$ mm, $Re = 10^5$, $L = 100$ m, $\rho = 998$ kg/m^3. Semilla: fórmula explícita de Haaland, $x_0 = -1.8\log_{10}\left[\left(\dfrac{\varepsilon/D}{3.7}\right)^{1.11} + \dfrac{6.9}{Re}\right]$.

Ejemplo 1: código en Python

```
import numpy as np
D, eps, Re = 0.050, 0.045e-3, 1.0e5 # m, m, adimensional
rr = eps / D # rugosidad relativa

F = lambda x: x + 2*np.log10(rr/3.7 + 2.51*x/Re)
dF = lambda x: 1 + (2/np.log(10)) * (2.51/Re) / (rr/3.7 + 2.51*x/Re)

x0 = -1.8*np.log10((rr/3.7)**1.11 + 6.9/Re) # semilla de Haaland
x, hist = newton_1d(F, dF, x0) # funcion de la diapositiva anterior
f = 1 / x**2 # factor de friccion de Darcy

V = Re*1.0e-3/(998*D) # velocidad [m/s] (mu = 1e-3 Pa s)
dP = f * (100/D) * 998 * V**2 / 2 # caida de presion en 100 m [Pa]
```

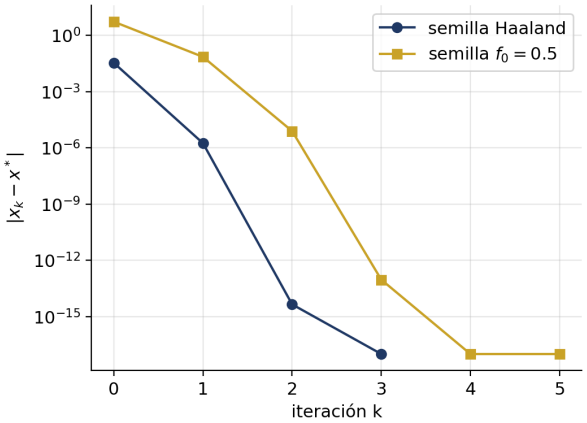
- Solo cambian `F`, `dF` y `x0`: el algoritmo es el mismo.
- Resultado: $f = 0.021832$, $V = 2.00$ m/s, $\Delta P \approx 87.5$ kPa en 100 m.

Ejemplo 1: resultados

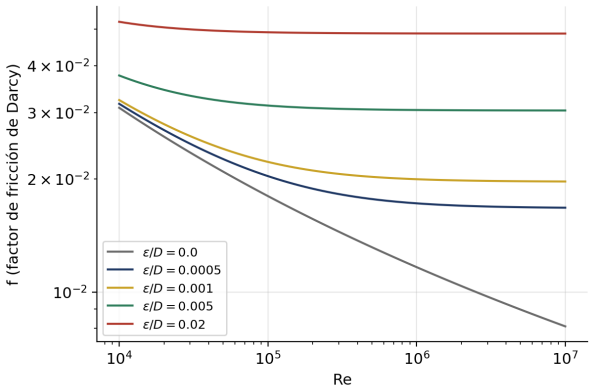
k	$x_k = 1/\sqrt{f_k}$	f_k	$\lvert F(x_k)\rvert$
0	6.8019806	0.021614	3.6×10^{-2}
1	6.7678533	0.021832	1.9×10^{-6}
2	6.7678551	0.021832	4.4×10^{-15}

Comparación de métodos (misma tolerancia):

Método	Iteraciones
Newton (semilla Haaland)	3
Newton ($f_0 = 0.5$, semilla mala)	5
Secante (`scipy.optimize.newton`)	4
`brentq` (intervalo $[3, 12]$)	5
Bisección (intervalo $[3, 12]$)	47



Ejemplo 1: Newton genera el diagrama de Moody



Repetimos Newton para 80 valores de Re entre 10^4 y 10^7 y para cinco rugosidades relativas ε/D entre 0 y 0.02.

Resultado

Con la semilla de Haaland, Newton necesitó como máximo 3 iteraciones en los 400 casos (tolerancia relativa 10^{-12} en el paso).

Una fórmula explícita aproximada (Haaland) como semilla + Newton para pulir: patrón muy común en ingeniería.

Ejemplo 1: la elección de la variable importa

Si aplicamos Newton directamente sobre f (en vez de $x = 1/\sqrt{f}$):

Semilla f_0	Iteraciones	Resultado
0.001	9	converge a $f = 0.021832$
0.02	5	converge a $f = 0.021832$
0.05	—	falla: un iterado cae en $f < 0$ y $\sqrt{f}$ no existe

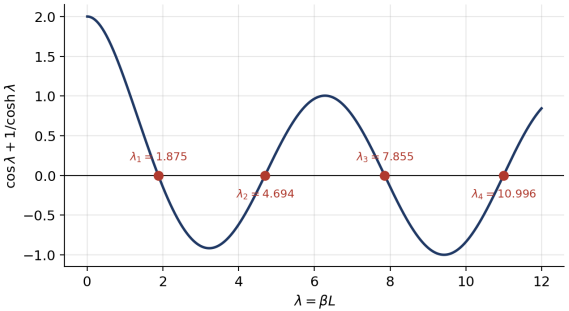
Lección

Newton funciona mejor cuando la función es *casi lineal* cerca de la raíz. Un buen cambio de variable puede ser la diferencia entre converger o fallar.

Ejemplo 2 (Vibraciones): frecuencias de una viga en voladizo

Viga de Euler-Bernoulli empotrada-libre: las frecuencias naturales salen de la ecuación característica

$$1 + \cos\lambda\,\cosh\lambda = 0, \qquad \lambda_n = \beta_n L, \qquad \omega_n = \lambda_n^2\sqrt{\frac{EI}{\rho A L^4}}.$$



Datos (acero): $E = 200$ GPa, $\rho = 7850$ kg/m^3, $L = 0.5$ m, sección $b \times h = 20 \times 5$ mm $\Rightarrow I = bh^3/12 = 2.083 \times 10^{-10}$ m^4, $A = 1.0 \times 10^{-4}$ m^2.

Forma escalada (evita el desbordamiento de cosh): dividimos entre $\cosh\lambda$,

$$g(\lambda) = \cos\lambda + \frac{1}{\cosh\lambda}, \quad g'(\lambda) = -\sin\lambda - \frac{\tanh\lambda}{\cosh\lambda}.$$

Hay infinitas raíces: ¿a cuál converge Newton?

Ejemplo 2: código y resultados

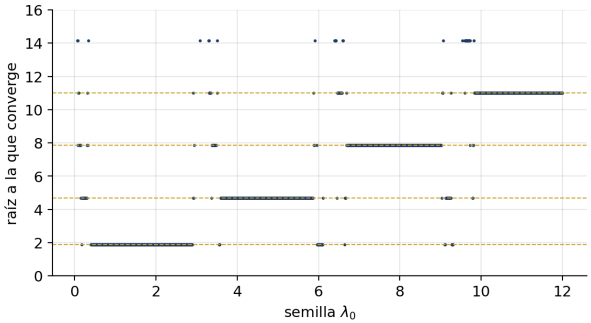
```
g = lambda l: np.cos(l) + 1/np.cosh(l)
dg = lambda l: -np.sin(l) - np.tanh(l)/np.cosh(l)

E, rho, L, b, h = 200e9, 7850.0, 0.5, 0.02, 0.005
I, A = b*h**3/12, b*h
for l0 in [2.0, 4.7, 7.9]: # semillas cerca de cada raiz
    lam, _ = newton_1d(g, dg, l0)
    w = lam**2 * np.sqrt(E*I/(rho*A*L**4)) # frecuencia natural [rad/s]
    print(lam, w, w/(2*np.pi))
```

Modo	λ_n	ω_n [rad/s]	f_n [Hz]
1	1.8751	102.46	16.31
2	4.6941	642.13	102.20
3	7.8548	1797.98	286.16

Las raíces de referencia se verificaron con `scipy.optimize.brentq`.

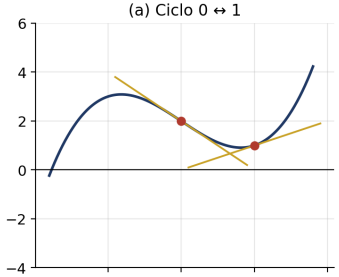
Ejemplo 2: la semilla decide a qué raíz converge Newton

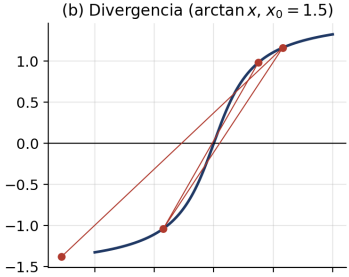


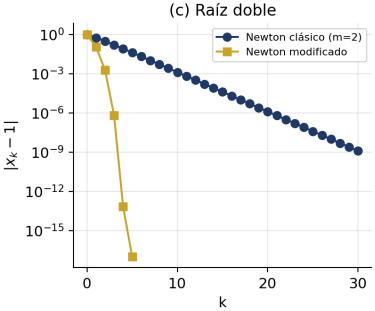
λ_0	Raíz	Iter.
1.0	1.8751	5
2.0	1.8751	5
3.0	-1.8751	7
4.0	4.6941	5
5.0	4.6941	4
6.5	10.9955	4
8.0	7.8548	4

Moraleja: Newton siempre converge a *alguna* raíz cercana, pero no necesariamente a la que usted quiere (aquí, $\lambda_0 = 3$ lleva a una raíz negativa sin sentido físico y $\lambda_0 = 6.5$ salta hasta la cuarta raíz). Use la **gráfica** para elegir semillas.

¿Cuándo falla Newton? Tres casos clásicos







(a) Ciclo. $f = x^3 - 2x + 2$, $x_0 = 0$: los iterados alternan $0, 1, 0, 1, \ldots$ para siempre.

(b) Divergencia. $f = \arctan x$ converge si $|x_0| < 1.39$; con $x_0 = 1.5$ los iterados *se alejan* de la raíz.

(c) Raíz múltiple. $f'(x^*) = 0$: la convergencia se vuelve lineal, con factor $\frac{m-1}{m}$.

Raíces múltiples y Newton modificado

Para $f(x) = (x-1)^2(x+2)$ (raíz doble en $x^* = 1$), con $x_0 = 2$:

k	$\lvert e_k \rvert$ clásico	e_{k+1}/e_k	$\lvert e_k \rvert$ modificado
0	1.0	0.556	1.0
1	0.556	0.536	1.1×10^{-1}
2	0.298	0.522	1.9×10^{-3}
3	0.155	0.512	6.3×10^{-7}
4	0.080	0.506	6.7×10^{-14}

Newton modificado (multiplicidad m conocida)

$$x_{k+1} = x_k - m\,\frac{f(x_k)}{f'(x_k)}$$

Recupera la convergencia cuadrática.

Con Newton clásico el error se reduce a la mitad por iteración ($m = 2$): necesita ~ 30 iteraciones para 10^{-9}.

Variantes cuando la derivada es incómoda o costosa

Método	Usa f'	Orden	Evaluaciones/iter.	Requiere
Bisección	No	1 (factor $\frac{1}{2}$)	1 f	intervalo con cambio de signo
Secante	No	≈ 1.618	1 f	dos semillas
Newton-Raphson	Sí	2	1 f + 1 f'	semilla, $f' \neq 0$
Newton modificado	Sí	2 (raíz múltiple)	1 f + 1 f'	multiplicidad m

Secante: reemplaza $f'(x_k)$ por una pendiente numérica

$$x_{k+1} = x_k - f(x_k)\,\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

Práctica profesional: combinar un método cerrado (robusto) con uno abierto (rápido), como hace `scipy.optimize.brentq`. En `scipy.optimize.newton`, si no se da `fprime`, se usa la secante.

Síntesis: Newton-Raphson en una variable

Ventajas

- Convergencia cuadrática cerca de la raíz simple.
- Muy pocas iteraciones (3–6 en nuestros ejemplos).
- Una fórmula, fácil de programar.

Limitaciones

- Necesita f' y una buena semilla.
- Puede ciclar, divergir o ir a otra raíz.
- Raíz múltiple $\Rightarrow$ convergencia lineal.

Recomendaciones prácticas

1. Grafique f antes de elegir la semilla.
2. Considere un cambio de variable que linealice el problema.
3. Controle $k_{\text{máx}}$ y verifique $|f(x_k)|$ y $|\Delta x_k|$.
4. Si Newton es inestable, use un método híbrido (`brentq`).

Contenido

1. Motivación y planteamiento
2. Newton-Raphson en una variable
3. Newton-Raphson para sistemas
4. Cierre

De una ecuación a n ecuaciones

Sistema de n ecuaciones no lineales con n incógnitas:

$$\mathbf{F}(\mathbf{x}) = \begin{pmatrix} F_1(x_1, \ldots, x_n) \\ \vdots \\ F_n(x_1, \ldots, x_n) \end{pmatrix} = \mathbf{0}, \qquad \mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}.$$

Matriz Jacobiana

$$\mathbf{J}(\mathbf{x}) = \begin{pmatrix} \dfrac{\partial F_1}{\partial x_1} & \cdots & \dfrac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial F_n}{\partial x_1} & \cdots & \dfrac{\partial F_n}{\partial x_n} \end{pmatrix}$$

La Jacobiana es la generalización de $f'(x)$: describe cómo cambia $\mathbf{F}$ al mover cada incógnita.

Deducción: Taylor multivariable

Aproximación lineal de $\mathbf{F}$ alrededor de $\mathbf{x}_k$:

$$\mathbf{F}(\mathbf{x}) \approx \mathbf{F}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)\,(\mathbf{x} - \mathbf{x}_k).$$

Igualando a cero en $\mathbf{x} = \mathbf{x}_{k+1}$ y llamando $\Delta\mathbf{x}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$:

$$\boxed{\mathbf{J}(\mathbf{x}_k)\,\Delta\mathbf{x}_k = -\mathbf{F}(\mathbf{x}_k), \qquad \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}_k}$$

Importante: no se invierte $\mathbf{J}$

En la práctica se *resuelve* el sistema lineal $n \times n$ en cada iteración (`np.linalg.solve`). Calcular $\mathbf{J}^{-1}$ explícitamente es más costoso y menos preciso.

Comparación

$n = 1$: $f'(x_k)\,\Delta x = -f(x_k) \;\Rightarrow\; \Delta x = -f/f'$ (el caso que ya conocemos).

Algoritmo y criterios de parada

Algoritmo

1. Elegir $\mathbf{x}_0$, tolerancia, $k_{\text{máx}}$.
2. Para $k = 0, 1, \ldots$
   1. Evaluar $\mathbf{F}(\mathbf{x}_k)$ y $\mathbf{J}(\mathbf{x}_k)$.
   2. Resolver $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$.
   3. $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}$.
   4. Parar si $\|\Delta\mathbf{x}\|$ (o $\|\mathbf{F}\|$) es pequeña.

Criterios con normas

- $\|\Delta\mathbf{x}_k\|_2 < \text{tol} \cdot \max(1, \|\mathbf{x}_{k+1}\|_2)$
- $\|\mathbf{F}(\mathbf{x}_{k+1})\|_2 < \text{tol}_F$

Cuidado con las unidades: si las incógnitas o las ecuaciones tienen unidades distintas (ángulos y longitudes, K y W), considere *escalar* antes de comparar normas.

Implementación en Python

```
def newton_nd(F, J, x0, tol=1e-12, maxit=50):
    x = np.array(x0, dtype=float)
    hist = [x.copy()]
    for k in range(maxit):
        dx = np.linalg.solve(J(x), -F(x)) # resolver J dx = -F (sin invertir J)
        x = x + dx
        hist.append(x.copy())
        if np.linalg.norm(dx) < tol * max(1.0, np.linalg.norm(x)):
            break
    return x, np.array(hist)
```

- `F(x)` devuelve un vector de tamaño n; `J(x)` devuelve una matriz $n \times n$.
- Es el mismo esqueleto que `newton_1d`: solo cambia la división por `solve`.

Ejemplo a mano: intersección circunferencia–parábola

$$F_1 = x^2 + y^2 - 4 = 0, \qquad F_2 = y - x^2 = 0, \qquad \mathbf{J} = \begin{pmatrix} 2x & 2y \\ -2x & 1 \end{pmatrix}, \qquad \mathbf{x}_0 = (1,1)^T.$$

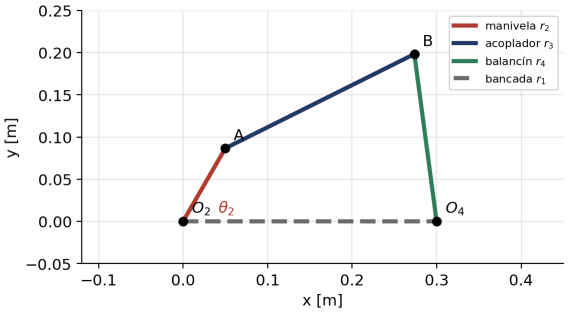
Primera iteración: $\mathbf{F}(\mathbf{x}_0) = (-2,\,0)^T$, $\mathbf{J}(\mathbf{x}_0) = \begin{pmatrix} 2 & 2 \\ -2 & 1 \end{pmatrix}$. Resolver $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F} = (2,\,0)^T$:

$$2a + 2b = 2, \;\; -2a + b = 0 \;\Rightarrow\; a = \tfrac{1}{3},\; b = \tfrac{2}{3} \;\Rightarrow\; \mathbf{x}_1 = \left(\tfrac{4}{3},\,\tfrac{5}{3}\right)^T \approx (1.3333,\,1.6667)^T.$$

k	x_k	y_k	$\|\mathbf{F}(\mathbf{x}_k)\|$
0	1.00000	1.00000	2.0
1	1.33333	1.66667	5.7×10^{-1}
2	1.25321	1.56410	1.8×10^{-2}
3	1.24963	1.56155	2.3×10^{-5}
4	1.24962	1.56155	4.9×10^{-11}

Solución exacta: $y = \frac{-1+\sqrt{17}}{2} = 1.56155$, $\quad x = \sqrt{y} = 1.24962$.

Ejemplo 3 (Mecanismos): cuatro barras — análisis de posición



Dado el ángulo de la manivela θ_2, hallar θ_3 (acoplador) y θ_4 (balancín). Cierre del lazo vectorial:

$$F_1 = r_2\cos\theta_2 + r_3\cos\theta_3 - r_4\cos\theta_4 - r_1 = 0$$
$$F_2 = r_2\sin\theta_2 + r_3\sin\theta_3 - r_4\sin\theta_4 = 0$$

Datos: $r_1 = 0.30$, $r_2 = 0.10$, $r_3 = 0.25$, $r_4 = 0.20$ m. Grashof: $s + l = 0.40 < p + q = 0.45$ (manivela-balancín).

$$\mathbf{J} = \begin{pmatrix} -r_3\sin\theta_3 & r_4\sin\theta_4 \\ r_3\cos\theta_3 & -r_4\cos\theta_4 \end{pmatrix}, \qquad \det\mathbf{J} = r_3 r_4\sin(\theta_3 - \theta_4).$$

Si $\theta_3 = \theta_4$ (barras alineadas), $\mathbf{J}$ es singular: es una posición de singularidad del mecanismo.

Ejemplo 3: código en Python

```
r1, r2, r3, r4 = 0.30, 0.10, 0.25, 0.20 # m

def make_system(th2):
    def F(u):
        th3, th4 = u
        return np.array([r2*np.cos(th2) + r3*np.cos(th3) - r4*np.cos(th4) - r1,
                         r2*np.sin(th2) + r3*np.sin(th3) - r4*np.sin(th4)])
    def J(u):
        th3, th4 = u
        return np.array([[-r3*np.sin(th3), r4*np.sin(th4)],
                         [ r3*np.cos(th3), -r4*np.cos(th4)]])
    return F, J

F, J = make_system(np.radians(60))
u0 = np.radians([20.0, 90.0]) # semilla [th3, th4] (rad)
u, hist = newton_nd(F, J, u0)
print(np.degrees(u)) # -> [26.5278 97.5622]
```

Los ángulos se trabajan en radianes dentro del código y se convierten a grados solo para imprimir.

Ejemplo 3: resultados para $\theta_2 = 60^\circ$

k	θ_3 [°]	θ_4 [°]	$\|\mathbf{F}\|$
0	20.0000	90.0000	3.2×10^{-2}
1	26.8027	97.2275	1.9×10^{-3}
2	26.5284	97.5617	3.6×10^{-6}
3	26.5278	97.5622	1.3×10^{-11}

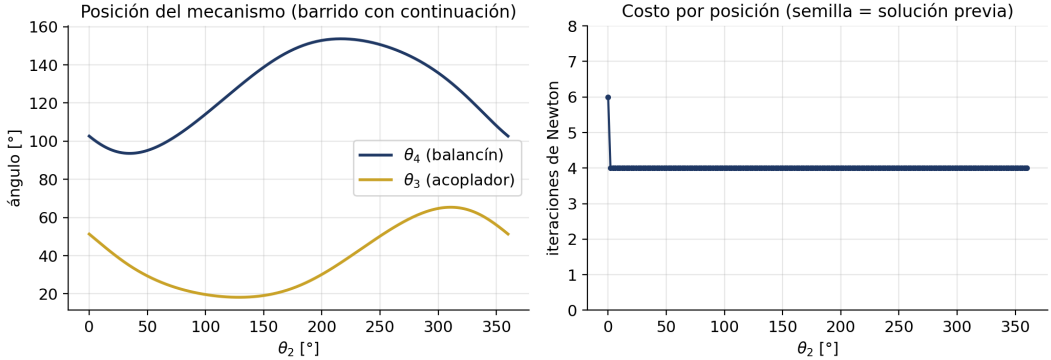
Comparación: `scipy.optimize.fsolve` con la misma Jacobiana obtiene $\theta_3 = 26.5278^\circ$, $\theta_4 = 97.5622^\circ$.

Dos ensambles, dos soluciones

Con la semilla $(-20^\circ, -90^\circ)$ Newton converge a $\theta_3 = -64.7410^\circ$, $\theta_4 = -135.7754^\circ$: la configuración *cruzada* del mismo mecanismo.

La semilla elige el ensamble del mecanismo. Para cada θ_2 hay dos soluciones geométricamente válidas.

Ejemplo 3: barrido de la manivela con continuación



Continuación: para cada nuevo θ_2 (paso de 2°) se usa como semilla la solución del ángulo anterior. Resultado: 4 iteraciones en promedio por posición, y se sigue la misma rama durante toda la vuelta. El balancín oscila entre $\theta_4 = 93.6^\circ$ y 153.6°.

Ejemplo 4 (Transferencia de calor): aleta con convección y radiación

Aleta cilíndrica de acero: $k = 50$ W/m K, $D = 10$ mm, $L = 150$ mm, base a $T_0 = 600$ K, $h = 15$ W/m^{2}K, $\varepsilon = 0.8$, $T_\infty = T_{sur} = 300$ K. Discretizamos con $n = 3$ nodos ($\Delta x = 50$ mm) *(modelo grueso, con fines didácticos)*.

$$F_i = \frac{kA}{\Delta x}(T_{i-1} - 2T_i + T_{i+1}) - hP\Delta x\,(T_i - T_\infty) - \varepsilon\sigma P\Delta x\,(T_i^4 - T_{sur}^4) = 0, \quad i = 1, 2$$

$$F_3 = \frac{kA}{\Delta x}(T_2 - T_3) - \left(\tfrac{P\Delta x}{2} + A\right)\left[h(T_3 - T_\infty) + \varepsilon\sigma(T_3^4 - T_{sur}^4)\right] = 0$$

Jacobiana tridiagonal (cada nodo solo depende de sus vecinos):

$$\frac{\partial F_i}{\partial T_i} = -\frac{2kA}{\Delta x} - hP\Delta x - 4\varepsilon\sigma P\Delta x\, T_i^3, \qquad \frac{\partial F_i}{\partial T_{i\pm1}} = \frac{kA}{\Delta x}$$

(el nodo 3 usa $-\frac{kA}{\Delta x} - (\frac{P\Delta x}{2} + A)(h + 4\varepsilon\sigma T_3^3)$ en la diagonal). Aquí $P = \pi D$, $A = \pi D^2/4$.

Ejemplo 4: código y semilla

```
sigma = 5.670e-8
k, D, L, h, eps, Tinf, T0 = 50.0, 0.010, 0.15, 15.0, 0.8, 300.0, 600.0
dx = L/3; P = np.pi*D; Ac = np.pi*D**2/4; kA = k*Ac/dx

def F(T):
    T1, T2, T3 = T
    w3 = P*dx/2 + Ac
    return np.array([
        kA*(T0-2*T1+T2) - h*P*dx*(T1-Tinf) - eps*sigma*P*dx*(T1**4-Tinf**4),
        kA*(T1-2*T2+T3) - h*P*dx*(T2-Tinf) - eps*sigma*P*dx*(T2**4-Tinf**4),
        kA*(T2-T3) - w3*(h*(T3-Tinf) + eps*sigma*(T3**4-Tinf**4))])

def J(T): # Jacobiana tridiagonal
    w3 = P*dx/2 + Ac
    d = lambda Ti, w: -2*kA - h*w - 4*eps*sigma*w*Ti**3
    return np.array([[d(T[0], P*dx), kA, 0],
                     [kA, d(T[1], P*dx), kA],
                     [0, kA, -kA - w3*(h + 4*eps*sigma*T[2]**3)]])

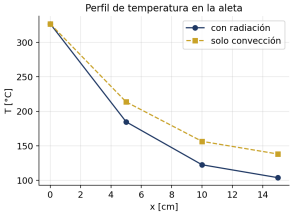
T, hist = newton_nd(F, J, np.full(3, T0)) # semilla: todos a la temperatura de la base
```

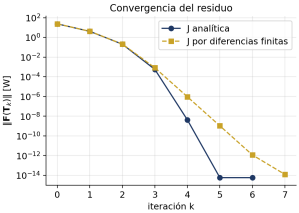
Ejemplo 4: resultados

k	T_1 [K]	T_2 [K]	T_3 [K]
0	600.000	600.000	600.000
1	484.274	443.345	432.856
2	459.114	398.141	380.017
3	457.863	395.605	376.895
4	457.860	395.598	376.886

$\|\mathbf{F}\|$ cae de 23.9 a 4.3×10^{-9} W en 4 iteraciones.

- La radiación baja las temperaturas $\approx$ 29–34 K respecto a solo convección (486.76, 429.55, 411.20 K).
- Calor que entra a la aleta: 11.164 W $=$ 6.968 W (convección) $+$ 4.195 W (radiación): el balance de energía cierra.





Jacobiana numérica: cuando derivar a mano es tedioso

Si $\mathbf{J}$ es difícil de derivar, se aproxima columna por columna con diferencias finitas:

$$\mathbf{J}_{:,j} \approx \frac{\mathbf{F}(\mathbf{x} + \delta\, \mathbf{e}_j) - \mathbf{F}(\mathbf{x})}{\delta}, \qquad j = 1, \ldots, n.$$

Costo: n evaluaciones extra de $\mathbf{F}$ por iteración.

Aleta ($\delta = 1$ K):

- Diferencia relativa máxima entre $\mathbf{J}$ analítica y numérica en la solución: 4.3×10^{-4}.
- Newton con $\mathbf{J}$ numérica: 7 iteraciones (vs. 6 con la analítica).
- Misma solución en todos los dígitos mostrados.

Ojo

Con $\mathbf{J}$ aproximada se pierde un poco la convergencia cuadrática (se vuelve *superlineal*), pero Newton sigue siendo muy eficiente. Es lo que hace `fsolve` internamente cuando no se le da `fprime`.

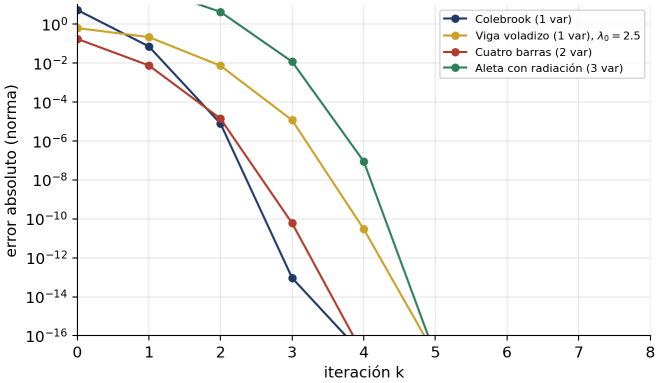
Comparación con `scipy.optimize`

Problema	Newton propio	`fsolve` con **J**	`fsolve` sin **J**
Cuatro barras ($\theta_2 = 60°$)	5 iter.	10 eval. de **F**	—
Aleta con radiación (3 nodos)	6 iter. (analítica) / 7 (numérica)	14 eval. de **F**	17 eval. de **F**

Iteraciones de Newton contadas hasta $\|\Delta\mathbf{x}\| < 10^{-13}$; el residuo ya es muy pequeño antes (en la aleta, 4.3×10^{-9} W tras 4 iteraciones).

- `fsolve` y `root(method='hybr')` usan el método híbrido de Powell (MINPACK): combina pasos de Newton con una región de confianza. Es más robusto que Newton puro, a costa de más evaluaciones de **F**.
- Nuestro Newton hace 1 evaluación de **F** y 1 de **J** por iteración.
- En la práctica: use `scipy.optimize.root`; programe Newton usted mismo para entender qué hace.

Convergencia cuadrática en los cuatro ejemplos



Estimación numérica del orden $p_k = \dfrac{\ln(e_{k+1}/e_k)}{\ln(e_k/e_{k-1})}$:

Ejemplo	p_k
Colebrook (1 var.)	2.1, 2.0
Viga (1 var.)	3.2, 1.9, 2.0
Cuatro barras (2 var.)	2.0, 2.0
Aleta (3 var.)	2.0, 2.0, 2.0

Con 1, 2 o 3 variables: el mismo comportamiento cuadrático.

Dificultades en sistemas y remedios

Problemas típicos

- Jacobiana singular o mal condicionada (p. ej. $\det \mathbf{J} = r_3 r_4 \sin(\theta_3 - \theta_4) \to 0$ en el mecanismo).
- Semilla lejana: converge a otra solución o diverge.
- Coste de armar y factorizar $\mathbf{J}$ en sistemas grandes ($n \sim 10^4$ o más).
- Ecuaciones con escalas muy distintas.

Remedios

- **Continuación:** usar la solución anterior como semilla (como en el barrido del mecanismo).
- **Newton amortiguado:** $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha\, \Delta\mathbf{x}_k$ con $0 < \alpha \leq 1$ (búsqueda lineal).
- **Cuasi-Newton** (Broyden): actualiza una aproximación de $\mathbf{J}$ sin recalcularla.
- Escalar variables y ecuaciones.
- Aprovechar la dispersión de $\mathbf{J}$ (matrices ralas).

Contenido

1 Motivación y planteamiento

2 Newton-Raphson en una variable

3 Newton-Raphson para sistemas

4 Cierre

Resumen: una idea, muchos problemas

	Una variable	**Sistema de n variables**
Problema	$f(x) = 0$	$\mathbf{F}(\mathbf{x}) = \mathbf{0}$
"Derivada"	$f'(x)$	$\mathbf{J}(\mathbf{x})$ (matriz $n \times n$)
Paso	$\Delta x = -f/f'$	$\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$
Actualización	$x_{k+1} = x_k + \Delta x$	$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}$
Convergencia	cuadrática (raíz simple)	cuadrática ($\mathbf{J}$ no singular)
Falla si	$f'(x^*) = 0$, semilla mala	$\det \mathbf{J} = 0$, semilla mala

Idea central

Linealizar, resolver el problema lineal, y repetir. En cada ejemplo de hoy cambió la *física*; el *algoritmo* fue el mismo.

Buenas prácticas

1. **Entienda el problema antes de programar:** grafique, estime órdenes de magnitud, busque las raíces físicamente razonables.
2. **Elija la semilla con criterio** (fórmula explícita aproximada, solución del paso anterior, gráfica).
3. **Verifique siempre el resultado:** sustituya en la ecuación original, revise el balance físico (por ejemplo, el balance de energía de la aleta).
4. **Registre el historial de iterados** y grafique el error: debe verse la caída cuadrática.
5. **Reporte la no convergencia** en vez de devolver un número sin sentido.
6. **Compare con una herramienta confiable** (`scipy.optimize`) para validar su implementación.

Ejercicios propuestos

1. **Colebrook.** Repita con $Re = 10^6$ y $\varepsilon/D = 0.002$. Compare Newton sobre $x = 1/\sqrt{f}$ y sobre f para distintas semillas.
2. **Viga.** Calcule las 5 primeras frecuencias de una viga de aluminio. Repita para viga empotrada-empotrada ($\cos\lambda \cosh\lambda = 1$).
3. **A mano.** Haga dos iteraciones de Newton para $x^3 - 2x - 5 = 0$ con $x_0 = 2$ y compare con el código.
4. **Raíz múltiple.** Estime numéricamente el orden de convergencia para $f(x) = (x-1)^3$ e implemente Newton modificado con $m = 3$.
5. **Cuatro barras.** Cambie $r_2 = 0.20$ m (ya no es Grashof). Encuentre los ángulos de θ_2 en los que $\mathbf{J}$ se vuelve singular y observe qué le pasa a Newton.
6. **Aleta.** Generalice el código a n nodos y compare la temperatura de la punta con $n = 3, 6, 12$. Pruebe semillas de 100 K, 1500 K y 3000 K: ¿cuántas iteraciones necesita? ¿Por qué converge aun con semillas lejanas?
7. **Newton amortiguado.** Implemente una búsqueda lineal (reducir α a la mitad hasta que $\|\mathbf{F}\|$ disminuya) y pruébela en un problema donde Newton puro falle.
8. **Propio.** Una masa colgada de dos resortes lineales fijos a dos puntos: plantee las ecuaciones de equilibrio en (x, y) con geometría no lineal, derive $\mathbf{J}$ y resuelva con Newton.

Referencias

- Chapra, S. C., Canale, R. P. *Métodos numéricos para ingenieros*. McGraw-Hill.
- Burden, R. L., Faires, J. D. *Numerical Analysis*. Cengage Learning.
- Kiusalaas, J. *Numerical Methods in Engineering with Python 3*. Cambridge University Press, 2013.
- Nocedal, J., Wright, S. J. *Numerical Optimization*. Springer, 2006 (Newton para sistemas, métodos globales).
- Colebrook, C. F. (1939). Turbulent flow in pipes, with particular reference to the transition region between the smooth and rough pipe laws. *J. Inst. Civil Eng.*, 11(4), 133–156.
- Haaland, S. E. (1983). Simple and explicit formulas for the friction factor in turbulent pipe flow. *J. Fluids Eng.*, 105(1), 89–90.
- Rao, S. S. *Mechanical Vibrations*. Pearson.
- Norton, R. L. *Design of Machinery*. McGraw-Hill.
- Incropera, F. P., DeWitt, D. P., Bergman, T. L., Lavine, A. S. *Fundamentals of Heat and Mass Transfer*. Wiley.
- Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

¿Preguntas?

Código y figuras: carpeta `scripts/` (ejecute `python run_all.py`).